

로드밸런싱 구축

오픈소스 로드밸런싱 시스템 구축 결과 보고서

VirtualBox + Ubuntu 22.04 + HAProxy 2.8 기반

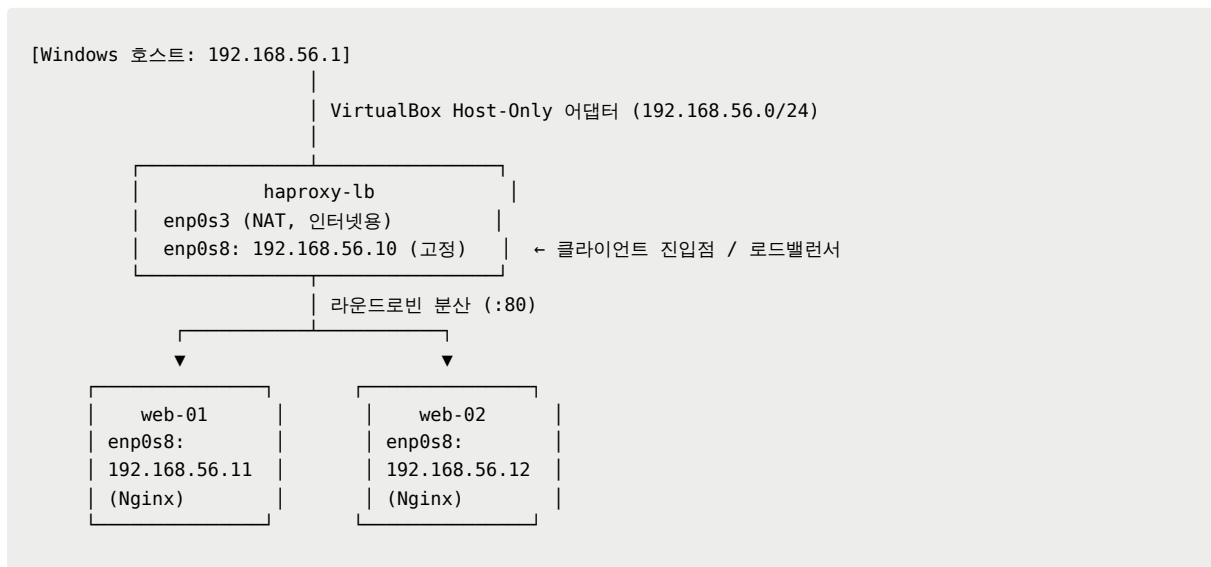
작성일: 2026-07

1. 개요

사내 신규 서비스 배포 시 단일 웹 서버로는 트래픽 증가와 장애 상황에 대응하기 어렵다는 문제 인식에서, 상용 로드밸런서 도입 전 검증 단계로 오픈소스 기반 로드밸런싱 환경을 로컬 가상화(VirtualBox) 환경에 직접 구축하였다. 로드밸런서로는 능동 헬스체크가 OSS 버전에서 기본 제공되고 VM 리소스 대비 처리 효율이 우수한 **HAProxy 2.8 LTS**를 채택하였으며, 백엔드 웹 서버는 Nginx로 구성하였다. VM 3대(로드밸런서 1대, 백엔드 웹 서버 2대)로 최소 구성을 완성하였고, 부하 분산 및 장애 복구(Failover) 동작을 실제로 확인하였다. 본 문서는 향후 동일 환경을 재현하거나 운영 환경으로 전환할 때 참고할 수 있도록 실제 수행한 절차와 시행착오를 그대로 남긴 것이다.

2. 아키텍처 요약

구성도



각 VM은 NAT(enp0s3, 인터넷/패키지 설치용)와 Host-Only(enp0s8, VM 간 통신용) 어댑터 2개를 사용한다. Windows 호스트에서는 NAT 포트포워딩을 통해 SSH로 각 VM에 접속한다.

VM 사양표

VM 이름	역할	CPU	RAM	Disk	비고
haproxy-lb	로드밸런서	2	2048 MB	20 GB	신규 생성, Ubuntu 22.04 Server 설치

VM 이름	역할	CPU	RAM	Disk	비고
web-01	백엔드 웹서버	2(계획 대비 변경)	2048 MB (계획 대비 변경)	20 GB	haproxy-lb를 Full Clone하여 생성
web-02	백엔드 웹서버	2 (계획 대비 변경)	2048 MB (계획 대비 변경)	20 GB	haproxy-lb를 Full Clone하여 생성

⚠ **추가 검증 필요:** 원래 계획(원본 메모 초반)은 web-01/web-02를 CPU 1 / RAM 1024MB로 신규 설치할 예정이었으나, 설치 중 반복된 키보드 freeze 문제로 인해 **haproxy-lb VM을 Full Clone하는 방식으로 전환**하였다(트러블슈팅 #2 참고). Full Clone은 원본 VM의 하드웨어 사양(CPU 2 / RAM 2048MB)을 그대로 복제하며, 원본 메모에는 복제 후 CPU/RAM을 상황에 맞게 별도로 축소 조정하기 가능하다. web-01/web-02의 **최종 CPU/RAM 값은 VirtualBox 설정 화면에서 직접 재확인**이 필요하다.

IP 체계표

Host	IP	비고
Windows 호스트	192.168.56.1	Host-Only 어댑터 게이트웨이
haproxy-lb	192.168.56.10	enp0s8 고정 IP
web-01	192.168.56.11	enp0s8 고정 IP
web-02	192.168.56.12	enp0s8 고정 IP

SSH 포트포워딩표 (Windows → VM)

VM	호스트 포트	게스트 포트	계정
haproxy-lb	2222	22	vboxuser / changeme
web-01	2223	22	vboxuser / changeme
web-02	2224	22	vboxuser / changeme

3. 사전 준비물

항목	내용	비고
가상화 소프트웨어	VirtualBox (실제 사용 버전 7.2.10)	다운로 드: https://www.virtualbox.org/wiki/Downloads
게스트 OS 이미지	Ubuntu 22.04.5 Server (amd64)	다운로 드: https://releases.ubuntu.com/22.04/ubuntu-22.04.5-live-server-amd64.iso
호스트 OS	Windows 11	관리자 권한 PowerShell 필요 (VBoxManage 사용 시)
최소 사양	haproxy-lb: 2 vCPU / 2GB RAM / 20GB Disk, web-0x: 1 vCPU / 1GB RAM / 20GB Disk (계획 기준)	실제로는 위 2절의 "추가 검증 필요" 항목 참고
기타 도구	socat (HAProxy Runtime API 소켓 조회용)	haproxy-lb에 <code>sudo apt install -y socat</code> 로 설치

4. 구축 절차

STEP 1 — Host-Only 네트워크 및 haproxy-lb VM 생성

(a) 목적: VM 간 통신용 Host-Only 네트워크를 확인하고 로드밸런서 VM을 생성한다.

(b) 실행 위치: Windows 호스트 (VirtualBox GUI)

(c) 설정값

항목	값
이름	haproxy-lb
ISO 이미지	ubuntu-22.04.5-live-server-amd64.iso
유형	Linux / Ubuntu (64-bit)
메모리	2048 MB
CPU	2
디스크	20 GB
자동 설치	"자동 설치" 체크 (필요시 수동 설치 진행)

네트워크 어댑터:

어댑터 1: 사용하기 체크 → NAT (인터넷/apt install용)
어댑터 2: 사용하기 체크 → 호스트 전용 어댑터 → VirtualBox Host-Only Ethernet Adapter

(d) 확인 방법: VM 생성 완료 후 목록에 haproxy-lb가 표시되고, 설정 → 네트워크에서 어댑터 1(NAT), 어댑터 2(Host-Only)가 모두 "사용하기"로 체크되어 있으면 완료.

STEP 2 — Ubuntu 22.04 Server 수동설치 (haproxy-lb)

⚠ 자동설치로 설치시 STEP 3으로 진행

(a) 목적: haproxy-lb VM에 OS를 설치한다.

(b) 실행 위치: haproxy-lb VM 콘솔 (VirtualBox 창)

(c) 설치 절차 (수동 설치 시도 기준)

```
1. Try or Install Ubuntu Server → Enter
2. 언어 선택: English → Enter
3. Installer update: Continue without updating → Enter
4. Keyboard configuration: Layout/Variant English (US) → Done
5. Choose type of install: Ubuntu Server (기본값) → Enter
6. Network connections: enp0s3(NAT) 자동 할당 확인, enp0s8은 그대로 둬 → Done
7. Proxy 설정: 비워두고 → Done
8. Mirror 설정: 그대로 → Done
9. Storage configuration: Use an entire disk → Done → Continue
10. Profile setup:
    Your name: admin
    Server name: haproxy-lb
    Username: admin
    Password: (설정)
    * admin으로 안될시 ubuntu
11. SSH Setup: Install OpenSSH server 체크(Space) → Done
```

12. Featured snaps: 선택 없이 → Done
13. 설치 완료 → Reboot Now

(d) 확인 방법: 재부팅 후 로그인 프롬프트가 뜨면 `hostname`, `ip a` 명령으로 haproxy-lb 이름과 enp0s3(NAT) IP 할당을 확인한다.

실제 결과: 위 수동 절차와 무관하게 최종적으로 Unattended Installation이 진행되어 계정 이 `vboxuser` / `changeme` 로 생성되었다(트러블슈팅 #1). 이후 모든 작업은 이 계정으로 진행하였다.

STEP 3 — SSH 포트포워딩 설정 및 haproxy-lb 접속

(a) 목적: VM 콘솔 대신 SSH로 편하게 작업하기 위해 포트포워딩을 구성한다.

(b) 실행 위치: Windows 호스트 (VirtualBox GUI) → 이후 Windows PowerShell

(c) 설정: VirtualBox 메인 화면 → haproxy-lb 우클릭 → 설정 → 네트워크 → 어댑터 1(NAT) → 고급 → 포트 포워딩 → [+]

이름	프로토콜	호스트 IP	호스트 포트	게스트 IP	게스트 포트
ssh	TCP	(빈칸)	2222	(빈칸)	22

저장 후 Windows PowerShell에서

```
ssh -p 2222 vboxuser@127.0.0.1
```

(d) 확인 방법: `vboxuser@haproxy-lb` 의 쉘 프롬프트가 뜨면 성공. (OpenSSH 미설치 상태라면 접속이 안 될 수 있으므로 트러블슈팅 #4 참고 — 콘솔에서 먼저 `sudo apt install -y openssh-server` 수행 필요)

STEP 4 — haproxy-lb 고정 IP 설정

(a) 목적: enp0s8(Host-Only)에 고정 IP 192.168.56.10을 할당한다.

(b) 실행 위치: haproxy-lb SSH 세션

(c) 명령어

```
ip a
# enp0s8 인터페이스가 보이는지, state DOWN인지 확인

sudo nano /etc/netplan/00-installer-config.yaml
```

파일 내용을 아래로 전체 교체 (들여쓰기는 스페이스 2칸, 탭 금지)

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: no
```

```
addresses:
- 192.168.56.10/24
```

저장(Ctrl+O → Enter → Ctrl+X) 후:

```
sudo netplan apply
ip a
```

(d) 확인 방법: `ip a` 출력에서 `enp0s8`에 `192.168.56.10/24`가 할당되어 있으면 완료. `netplan apply` 시 나오는 권한 관련 경고 메시지는 무해하므로 무시한다.

STEP 5 — web-01, web-02 VM 생성 (Full Clone 방식)

(a) 목적: 백엔드 웹 서버 VM 2대를 생성한다.

(b) 실행 위치: Windows 호스트 (VirtualBox GUI)

(c) 절차 (Ubuntu 신규 설치 반복 시도 중 키보드 freeze로 실패 → 아래 Clone 방식으로 전환하여 실제 진행)

1. haproxy-lb 우클릭 → Clone
2. 이름: web-01 (web-02는 동일 절차를 이름만 다르게 반복)
 - "OS Installation Options" 관련 옵션 둘 다 체크 해제
 - 스냅샷: Current Machine State
3. MAC Address Policy: 모든 네트워크 어댑터의 새 MAC 주소 생성 선택
4. Clone type: Full clone
5. Finish

(d) 확인 방법: VirtualBox 목록에 web-01, web-02가 각각 생성되고, 두 VM 모두 기동 가능한 상태이면 완료.

원래 계획은 web-01/web-02를 개별 신규 설치(CPU 1 / RAM 1024MB)하는 것이었으나 설치 중 Profile Setup 화면에서 키보드 freeze가 반복 발생하여 Full Clone 방식으로 전환하였다(트러블슈팅 #2).

STEP 6 — web-01, web-02 SSH 접속 및 hostname/IP 변경

(a) 목적: 복제된 VM은 haproxy-lb와 이름/IP가 동일하므로 각각 고유값으로 변경한다.

(b) 실행 위치: Windows PowerShell(SSH 접속) → web-01/web-02 SSH 세션(설정 변경)

(c) 포트포워딩 설정 (Windows, VirtualBox GUI)

VM	이름	프로토콜	호스트 포트	게스트 포트
web-01	ssh	TCP	2223	22
web-02	ssh	TCP	2224	22

web-01 SSH 접속 및 변경

```
ssh -p 2223 vboxuser@127.0.0.1
```

```

sudo hostnamectl set-hostname web-01
sudo nano /etc/netplan/00-installer-config.yaml
# 192.168.56.10 → 192.168.56.11 로 변경 후 저장(Ctrl+O → Enter → Ctrl+X)
sudo netplan apply
hostname
ip a

```

web-02 SSH 접속 및 변경

```
ssh -p 2224 vboxuser@127.0.0.1
```

```

sudo hostnamectl set-hostname web-02
sudo nano /etc/netplan/00-installer-config.yaml
# 192.168.56.10 → 192.168.56.12 로 변경 후 저장
sudo netplan apply
hostname
ip a

```

(d) 확인 방법: 각 VM에서 `hostname` 이 web-01/web-02로, `ip a` 의 enp0s8 IP가 192.168.56.11/192.168.56.12로 각각 표시되면 완료.

STEP 7 — Nginx 설치 및 헬스체크 엔드포인트 구성 (web-01, web-02)

(a) 목적: 백엔드 웹 서버와 HAProxy용 헬스체크(`/health`) 엔드포인트를 구성한다.

(b) 실행 위치: web-01, web-02 각 SSH 세션 (동시 진행 가능)

(c) 명령어

```

sudo apt update && sudo apt install -y nginx
echo "<h1>Backend: $(hostname)</h1>" | sudo tee /var/www/html/index.htm
l
sudo tee /etc/nginx/conf.d/health.conf <<'EOF'
server {
    listen 80 default_server;
    location /health {
        access_log off;
        return 200 "OK\n";
        add_header Content-Type text/plain;
    }
    location / {
        root /var/www/html;
        index index.html;
    }
}
EOF
sudo nginx -t

```

```
sudo systemctl enable nginx
sudo systemctl start nginx
```

기본 사이트 설정과의 `default_server` 충돌 제거(트러블슈팅 #7)

```
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -tsudo systemctl reload nginx
curl http://localhost/health
```

(d) 확인 방법

- web-01/web-02 각각에서 `curl http://localhost/health` → OK 응답
- haproxy-lb에서 `curl http://192.168.56.11/health`, `curl http://192.168.56.12/health` → 둘 다 OK 응답

STEP 8 — HAProxy 2.8 설치 (haproxy-lb)

(a) 목적: PPA를 통해 HAProxy 2.8을 설치한다.

(b) 실행 위치: haproxy-lb SSH 세션

(c) 명령어

```
sudo apt install -y software-properties-common
sudo add-apt-repository -y ppa:vbernat/haproxy-2.8
sudo apt update
sudo apt install -y haproxy=2.8.*
haproxy -v
```

(d) 확인 방법: `haproxy -v` 출력에 HAProxy 2.8 계열 버전이 표시되면 완료. 실제 설치된 버전은 **2.8.25**로 확인되었다.

STEP 9 — HAProxy 설정 파일 작성

(a) 목적: 로드밸런싱 규칙, 헬스체크, Stats 페이지를 정의한다.

(b) 실행 위치: haproxy-lb SSH 세션

(c) 명령어

```
sudo nano /etc/haproxy/haproxy.cfg
```

아래 내용으로 작성 (※ HTTP/1.0 관련 수정은 트러블슈팅 #8 적용 후 최종본)

```
global
    log /dev/log local0
    log /dev/log local1 notice
    chroot /var/lib/haproxy
    stats socket /run/haproxy/admin.sock mode 660 level admin expose-fd listeners
    stats timeout 30s
    user haproxy
    group haproxy
    daemon
    maxconn 20000

defaults
```

```

log      global
mode     http
option   httplog
option   dontlognull
timeout  connect 5s
timeout  client 30s
timeout  server 30s
timeout  http-request 10s

frontend stats
  bind *:8404
  stats enable
  stats uri /stats
  stats refresh 10s
  stats auth admin:changeme123

frontend http_front
  bind *:80
  default_backend web_servers
  option forwardfor

backend web_servers
  balance roundrobin
  option httpchk GET /health HTTP/1.0
  http-check expect status 200
  server web-01 192.168.56.11:80 check inter 2s rise 2 fall 3
  server web-02 192.168.56.12:80 check inter 2s rise 2 fall 3

```

작성 후 문법 검증:

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
```

(d) 확인 방법: 검증 명령이 오류 없이 통과(Configuration file is valid)하면 완료.

참고: 최초 작성 시 `option httpchk GET /health HTTP/1.1\r\nHost:\ localhost` 로 시도했으나 헬스체크가 실패했다(트러블슈팅 #8). 위 최종본은 HTTP/1.0으로 수정한 버전이다.

STEP 10 — HAProxy 기동 및 헬스체크 확인

(a) 목적: HAProxy를 기동하고 백엔드 서버 상태를 확인한다.

(b) 실행 위치: haproxy-lb SSH 세션

(c) 명령어

```
sudo systemctl status haproxy --no-pager
```

네트워크/응답 확인

```
ping -c 2 192.168.56.11
```

```
ping -c 2 192.168.56.12
```

```
curl http://192.168.56.11/health
```

```
curl http://192.168.56.12/health
```

socat 미설치 시

```
sudo apt install -y socat
```

```
# 서버 상태 조회
echo "show servers state" | sudo socat stdio /run/haproxy/admin.sock
```

(d) 확인 방법: `show servers state` 결과의 `srv_op_state` 값이 2(UP)이면 정상. (최초에는 HTTP/1.1 헬스체크 문제로 DOWN 상태였으며, HTTP/1.0으로 수정 후 재기동하여 UP으로 전환됨 — 트러블슈팅 #8)

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo systemctl restart haproxy
```

STEP 11 — 부하 분산 및 장애 복구 동작 검증

(a) 목적: 로드밸런싱 및 Failover 동작을 실제로 확인한다.

(b) 실행 위치: haproxy-lb SSH 세션 (부하분산/장애복구), Windows 브라우저 (Stats 페이지)

(c) 명령어

부하 분산 확인

```
for i in $(seq 1 6); do curl -s http://192.168.56.10/; done
```

Stats 페이지 확인 (Windows 브라우저)

```
http://192.168.56.10:8404/stats
ID: admin
PW: changeme123
```

장애 복구 테스트

```
# web-01에서 실행
sudo systemctl stop nginx
```

```
# haproxy-lb에서 약 6초 후(fall 3 × inter 2s) 실행
for i in $(seq 1 4); do curl -s http://192.168.56.10/; done
# → web-02로만 응답이 오면 성공
```

```
# web-01 복구
sudo systemctl start nginx
```

```
# web-02 중단 후 haproxy-lb에서 교차 확인
for i in $(seq 1 6); do curl -s http://192.168.56.10/; done
```

(d) 확인 방법: 6회 요청이 web-01/web-02로 교대 응답, web-01 중단 시 web-02로만 응답, 복구 후 정상 교대 응답으로 돌아오면 성공. Stats 페이지에서 admin/changeme123 로그인 후 두 서버의 Status가 확인 되면 완료.

5. 트러블슈팅

문제	원인	해결
VM 생성 시 "자동 설치 건너뛰기"를 체크했음에도 자동(Unattended) 설치로 진행됨	VirtualBox 7.x는 "Proceed with Unattended Installation" 옵션이 기본 활성화되어 있음	VM 삭제 후 재생성 시도했으나, haproxy-lb는 최종적으로 자동 설치로 진행되어 <code>vboxuser</code> / <code>changeme</code> 계정이 생성됨. 이후 이 계정으로 작업 진행
Ubuntu Server 수동 설치 중 Profile Setup 화면에서 키보드 입력이 멈춤(freeze)	VirtualBox 환경에서의 설치 화면 버그로 추정 (원본 메모 기준 원인 미확정)	web-01, web-02는 신규 설치 대신 haproxy-lb를 Full Clone하여 생성하는 방식으로 전환
Ubuntu 설치 중 Mirror 체크가 첫 시도에서 타임아웃됨	미확인 (네트워크 지연 등으로 추정)	오류 화면에서 Continue 선택하여 설치 진행
SSH 접속이 되지 않음	자동 설치(Unattended Install) 과정에 OpenSSH 서버가 포함되지 않음	VM 콘솔에서 <code>sudo apt install -y openssh-server</code> 수동 설치 후 접속 재시도
<code>systemctl status</code> 실행 시 화면이 멈춘 것처럼 보임	pager(less)가 실행되어 출력이 대기 상태가 됨	<code>--no-pager</code> 옵션을 붙여 실행 (<code>sudo systemctl status haproxy --no-pager</code>)
Nginx <code>/health</code> 엔드포인트가 404 또는 예상과 다르게 동작	<code>/etc/nginx/sites-enabled/default</code> 의 <code>default_server</code> 설정과 <code>health.conf</code> 의 <code>default_server</code> 설정이 충돌	<code>sudo rm /etc/nginx/sites-enabled/default</code> 실행 후 <code>nginx -t</code> → <code>systemctl reload nginx</code>
HAProxy 헬스체크가 계속 실패(서버가 DOWN으로 표시)	<code>option httpchk GET /health HTTP/1.1</code> 사용 시 HTTP/1.1은 Host 헤더가 필수인데 헬스체크 요청에 Host 헤더가 없어 nginx가 요청을 거부함	<code>option httpchk GET /health HTTP/1.0</code> 으로 변경 후 <code>haproxy -c -f</code> 로 검증, <code>systemctl restart haproxy</code> 로 재기동
<code>netplan apply</code> 실행 시 권한 관련 경고 메시지 출력	미확인 (netplan 내부 동작 특성으로 추정)	실제 네트워크 설정에는 영향이 없는 것으로 확인되어 무시하고 진행

6. 최종 검증 결과

검증 항목	방법	결과
부하 분산(Round Robin)	haproxy-lb에서 <code>for i in \$(seq 1 6); do curl -s http://192.168.56.10/; done</code> 실행	web-01, web-02가 교대로 응답하는 것을 확인 (daily_log 2026-07-01 기준). 다만 curl 응답 원문 로그는 별도로 보존되어 있지 않아, 본 문서에서는 daily_log에 기록된 확인 결과를 근거로 기재함
장애 복구(Failover)	web-01에서 <code>nginx</code> 중단 → 약 6초(fall 3 × inter 2s) 대기 → haproxy-lb에서 재요청	web-02로만 응답이 전환되는 것을 확인. web-01 복구(<code>nginx</code> 재시작) 후 다시 두 서버로 분산되는 것을 확인
헬스체크 상태	<code>echo "show servers state" sudo socat stdio /run/haproxy/admin.sock</code>	<code>srv_op_state = 2 (UP)</code> 로 두 서버 모두 정상 확인 (HTTP/1.0 수정 이후)
Stats 페이지	브라우저에서 <code>http://192.168.56.10:8404/stats</code> 접속, <code>admin</code> / <code>changeme123</code> 로그인	접근 및 로그인 성공 확인 (daily_log 2026-07-01 기준)

전체 항목이 확인되어 **2026-07-01 기준 구축 및 기본 동작 검증이 완료된 상태**이다

7. 결론 및 향후 개선 제안

VM 3대 규모의 최소 구성으로 HAProxy 기반 로드밸런싱과 Nginx 백엔드의 부하 분산/장애 복구 동작을 검증하였다. 다만 현재 구성은 검증(PoC) 목적의 로컬 환경이며, 운영 전환 시에는 아래 사항의 추가 검토가 필요하다.

1. **SSL/TLS 종단(Termination) 및 Stats 페이지 보안 강화:** 현재 HAProxy는 80번 포트로만 HTTP 트래픽을 처리하며, Stats 페이지(`:8404/stats`)의 인증 정보(`admin:changeme123`)도 예시용 비밀번호가 그대로 사용되고 있다. 운영 전환 시 인증서 적용 및 Stats 페이지 접근 IP 제한(또는 비밀번호 변경)이 필요하다.
2. **로드밸런서 자체의 이중화(HA):** 현재 haproxy-lb가 단일 장애점(SPOF)이다. keepalived(VRRP) 등을 이용한 Active-Standby 이중화 구성 검토가 필요하다. (본 구축 범위에는 포함되지 않았으며, 별도 검증 필요)
3. **IaC(Ansible/Terraform) 전환:** 현재는 VM 생성부터 설정까지 전 과정을 수동(GUI, SSH 수기 입력)으로 진행하였다. `haproxy.cfg` 를 Ansible Jinja2 템플릿으로 관리하고, VM 생성 자체는 Terraform(또는 Vagrant)으로 코드화하면 재현성과 변경 이력 관리가 개선될 것으로 판단된다.